

Proyecto TechDAM - JDBC Avanzado

TECHDAM

Marcos Pérez Ramírez

[GitHub](#)

16/11/2025

Índice

Enunciado	3
Parte 1: Base de Datos	3
Parte 2: Capturas de Pantalla	4
Tablas creadas en MySQL Workbench	4
Ejecución Exitosa de CRUD empleados.....	5
Invocación de procedimiento almacenado.....	8
Transacciones con commit y rollback.....	9
Pool de Conexiones configurado.....	11
Parte 3: Explicación de Decisiones Técnicas	13
Parte 4: Problemas Encontrados y Soluciones.....	14

Enunciado

Debes completar el desarrollo del sistema TechDAM implementando todas las funcionalidades requeridas y demostrando dominio de JDBC avanzado.

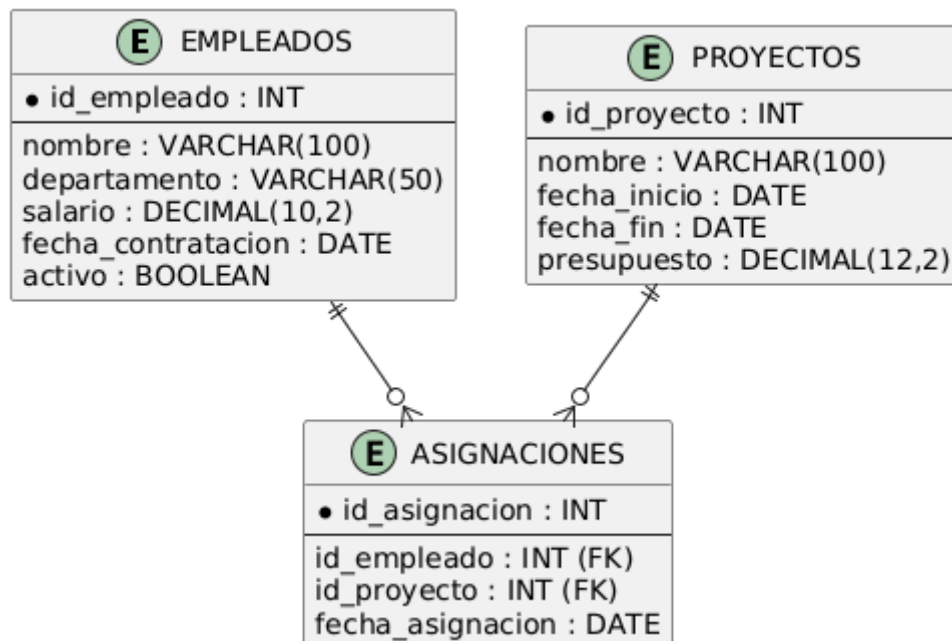
Parte 1: Base de Datos

Este es el diagrama Entidad Relación de mi base de datos.

Empleados: solo PK (no tiene FKs)

Proyectos: solo PK (no tiene FKs)

Asignaciones: PK propia (id_asignacion) y dos FKs (id_empleado, id_proyecto)



Parte 2: Capturas de Pantalla

Tablas creadas en MySQL Workbench

```
DROP DATABASE IF EXISTS techdam_completo;
CREATE DATABASE IF NOT EXISTS techdam_completo;
USE techdam_completo;

) CREATE TABLE empleados (
    id_empleado INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    departamento VARCHAR(50),
    salario DECIMAL(10,2),
    fecha_contratacion DATE,
    activo BOOLEAN DEFAULT TRUE
~ );

) CREATE TABLE proyectos (
    id_proyecto INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    fecha_inicio DATE,
    fecha_fin DATE,
    presupuesto DECIMAL(12,2)
~ );

) CREATE TABLE asignaciones (
    id_asignacion INT AUTO_INCREMENT PRIMARY KEY,
    id_empleado INT,
    id_proyecto INT,
    fecha_asignacion DATE,
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (id_proyecto) REFERENCES proyectos(id_proyecto)
        ON DELETE CASCADE ON UPDATE CASCADE
~ );
```

Output

Time	Action	Message
12:30:46	DROP DATABASE IF EXISTS techdam_completo	3 row(s) affected
12:30:46	CREATE DATABASE IF NOT EXISTS techdam_completo	1 row(s) affected
12:30:46	USE techdam_completo	0 row(s) affected
12:30:46	CREATE TABLE empleados (id_empleado INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, departamento VARCHA...	0 row(s) affected
12:30:46	CREATE TABLE proyectos (id_proyecto INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, fecha_inicio DATE, fech...	0 row(s) affected
12:30:46	CREATE TABLE asignaciones (id_asignacion INT AUTO_INCREMENT PRIMARY KEY, id_empleado INT, id_proyecto INT, fecha_asignacion D...	0 row(s) affected

Ejecución Exitosa de CRUD empleados.

Mostrar empleados

```
---- MENÚ PRINCIPAL ----
1. Gestión de empleados
2. Gestión de proyectos
3. Procedimientos almacenados
4. Transacciones
0. Salir
Elige opción: 1

--- Gestión de empleados ---
1. Mostrar empleados
2. Insertar empleado
3. Modificar empleado
4. Eliminar empleado
0. Volver
Elige opción: 1
[main] INFO com.zaxxer.hikari.HikariDataSource - DAMPool - Starting...
[main] INFO com.zaxxer.hikari.pool.HikariPool - DAMPool - Added connection com.mysql.cj.jdbc.ConnectionImpl@6c779568
[main] INFO com.zaxxer.hikari.HikariDataSource - DAMPool - Start completed.
Pool de conexiones HikariCP inicializado correctamente.
Empleado{idEmpleado=1, nombre='Ana Pérez', departamento='Desarrollo', salario=2500.00, fecha_contratacion=2020-03-01, activo=true}
Empleado{idEmpleado=2, nombre='Luis Gómez', departamento='Marketing', salario=2000.00, fecha_contratacion=2019-07-15, activo=true}
Empleado{idEmpleado=3, nombre='Marta López', departamento='Desarrollo', salario=2700.00, fecha_contratacion=2021-06-10, activo=true}
Empleado{idEmpleado=4, nombre='Carlos Díaz', departamento='Soporte', salario=1800.00, fecha_contratacion=2018-02-20, activo=true}
Empleado{idEmpleado=5, nombre='Sofía Ruiz', departamento='Desarrollo', salario=3000.00, fecha_contratacion=2017-11-05, activo=false}
```

Insertar empleados

```
--- Gestión de empleados ---
1. Mostrar empleados
2. Insertar empleado
3. Modificar empleado
4. Eliminar empleado
0. Volver
Elige opción: 2
---- Crear nuevo empleado ----
Nombre: Marcos Perez
Departamento: DAM
Salario: 1500
Fecha de contratación (yyyy-MM-dd): 2004-05-20
¿Activo? (true/false): true
Empleado creado con ID 6
```

Modificar empleado

```
--- Gestión de empleados ---
1. Mostrar empleados
2. Insertar empleado
3. Modificar empleado
4. Eliminar empleado
0. Volver
Elige opción: 3
Introduce el ID del empleado a modificar: 6
Nuevo nombre (Marcos Perez):
Nuevo departamento (DAM): DAW
Nuevo salario (1500.00): 1600
Fecha de contratación (yyyy-MM-dd) (2004-05-20):
¿Activo? (true/false) (true): false
Empleado modificado correctamente.
```

Eliminar empleado

```
--- Gestión de empleados ---
1. Mostrar empleados
2. Insertar empleado
3. Modificar empleado
4. Eliminar empleado
0. Volver
Elige opción: 4
Introduce el ID del empleado a eliminar: 6
Empleado eliminado correctamente.
```

Ejecución Exitosa de CRUD proyectos.

Mostrar proyectos

```
--- Gestión de proyectos ---
1. Mostrar proyectos
2. Insertar proyecto
3. Modificar proyecto
4. Eliminar proyecto
0. Volver
Elige opcion: 1
Proyecto{idProyecto=1, nombre='Sistema ERP', fecha_inicio=2023-01-10, fecha_fin=2023-12-31, presupuesto=50000.00}
Proyecto{idProyecto=2, nombre='App Móvil', fecha_inicio=2023-04-01, fecha_fin=2023-10-01, presupuesto=30000.00}
Proyecto{idProyecto=3, nombre='Campaña Publicitaria', fecha_inicio=2023-06-01, fecha_fin=2023-09-30, presupuesto=15000.00}
Proyecto{idProyecto=4, nombre='Portal Web', fecha_inicio=2022-02-15, fecha_fin=2022-12-30, presupuesto=20000.00}
Proyecto{idProyecto=5, nombre='CRM Interno', fecha_inicio=2024-01-01, fecha_fin=2024-12-31, presupuesto=45000.00}
```

Insertar proyecto

```
Elige opcion: 2
---- Crear nuevo proyecto ----
Nombre: Proyecto Marcos
Fecha inicio (yyyy-MM-dd): 2005-06-21
Fecha fin (yyyy-MM-dd): 2007-02-12
Presupuesto: 9000
Proyecto creado con ID 6
```

Modificar proyecto

```
Elige opcion: 3
Introduce el ID del proyecto a modificar: 6
Nuevo nombre (Proyecto Marcos):
Nueva fecha inicio (yyyy-MM-dd) (2005-06-21):
Nueva fecha fin (yyyy-MM-dd) (2007-02-12):
Nuevo presupuesto (9000.00): 12
Proyecto modificado correctamente.
```

Eliminar proyecto

```
Elige opcion: 4
Introduce el ID del proyecto a eliminar: 6
Proyecto eliminado correctamente.
```

Invocación de procedimiento almacenado.

Seleccionamos la opción de actualizar el salario por departamento.

```
--- Procedimientos almacenados ---
1. Actualizar salario por departamento
2. Empleados por Proyecto
0. Volver
Elige opcion: 1
Introduce el departamento: Desarrollo
Introduce el porcentaje de incremento (Ejemplo: 3.5 para 3.5%): 5
[main] INFO com.zaxxer.hikari.HikariDataSource - DAMPool - Starting...
[main] INFO com.zaxxer.hikari.pool.HikariPool - DAMPool - Added connection com.mysql.cj.jdbc.ConnectionImpl@68e965f5
[main] INFO com.zaxxer.hikari.HikariDataSource - DAMPool - Start completed.
Pool de conexiones HikariCP inicializado correctamente.
Empleados actualizados: 2
```

Antes:

	nombre	salario	departamento	dinero_departamento
▶	Ana Pérez	2500.00	Desarrollo	5200.00
	Marta López	2700.00	Desarrollo	5200.00
	Luis Gómez	2000.00	Marketing	2000.00
	Carlos Díaz	1800.00	Soporte	1800.00

Después:

	nombre	salario	departamento	dinero_departamento
	Ana Pérez	2625.00	Desarrollo	5460.00
	Marta López	2835.00	Desarrollo	5460.00
	Luis Gómez	2000.00	Marketing	2000.00
	Carlos Díaz	1800.00	Soporte	1800.00

Seleccionamos la opción para ver cuántos empleados hay asignados a un proyecto.

```
--- Procedimientos almacenados ---
1. Actualizar salario por departamento
2. Empleados por Proyecto
0. Volver
Elige opcion: 2
Introduce el ID del proyecto: 2
Total empleados asignados al proyecto: 1
```

	id_proyecto	proyecto	empleados_asignados
▶	1	Sistema ERP	1
	2	App Móvil	1
	3	Campaña Publicitaria	1
	4	Portal Web	1
	5	CRM Interno	1

Transacciones con commit y rollback.

Seleccionamos transferir presupuesto entre proyectos.

```
--- Transacciones ---
1. Transferir presupuesto entre proyectos
2. Asignar empleados a proyecto
0. Volver
Elige opcion:
```

Esta es la tabla sin haber tocado ningún dato

	id_proyecto	nombre	presupuesto
▶	1	Sistema ERP	50000.00
	2	App Móvil	30000.00
	3	Campaña Publicitaria	15000.00
	4	Portal Web	20000.00
	5	CRM Interno	45000.00

```
--- Transacciones ---
1. Transferir presupuesto entre proyectos
2. Asignar empleados a proyecto
0. Volver
Elige opcion: 1
Id de proyecto origen: 5
Id de proyecto destino: 1
Monto a transferir: 5000
Transferencia realizada correctamente.
```

	id_proyecto	nombre	presupuesto
	1	Sistema ERP	55000.00
	2	App Móvil	30000.00
	3	Campaña Publicitaria	15000.00
	4	Portal Web	20000.00
	5	CRM Interno	40000.00

Seleccionamos asignar empleados a proyecto.

Así está la tabla sin tocar los datos.

id_proyecto	proyecto	empleados_asignados
1	Sistema ERP	1
2	App Móvil	1
3	Campaña Publicitaria	1
4	Portal Web	1
5	CRM Interno	1

```
--- Transacciones ---  
1. Transferir presupuesto entre proyectos  
2. Asignar empleados a proyecto  
0. Volver  
Elige opcion: 2  
Id de proyecto destino: 5  
Introduce los IDs de empleados separados por coma: 1,2,3
```

Después de hacer esto, la tabla se varía de la siguiente manera.

Ahora vemos que en el proyecto con id 5 hay 4 empleados asignados.

id_proyecto	proyecto	empleados_asignados
1	Sistema ERP	1
2	App Móvil	1
3	Campaña Publicitaria	1
4	Portal Web	1
5	CRM Interno	4

Pool de Conexiones configurado

```
package config;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

import java.io.InputStream;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.Properties;

/**
 * Utilidad para gestionar un DataSource basado en HikariCP mediante
 * inicialización
 * estática. Esta clase expone métodos estáticos para obtener una Connection
 * y para
 * cerrar el pool cuando la aplicación deja de necesitarlo.
 * <p>
 * Propósito académico:
 * - Mostrar cómo inicializar un pool de conexiones al cargar la clase.
 * - Explicar la lectura de parámetros desde un fichero de propiedades.
 * - Enseñar la importancia de cerrar el pool al finalizar la aplicación.
 * <p>
 * Observaciones didácticas:
 * - La inicialización se realiza en un bloque static para disponer del pool
 * desde
 * el arranque de la aplicación.
 * - En entornos productivos, el fichero de propiedades no debe contener
 * credenciales
 * en texto plano y su carga debe incluir comprobaciones más exhaustivas.
 */
public class DatabaseConfig {
    // DataSource compartido por toda la aplicación
    private static HikariDataSource dataSource;

    /**
     * Bloque de inicialización estático:
     * Se ejecuta al cargar la clase y configura el pool leyendo propiedades
     * desde
     * el recurso 'db.properties' situado en el classpath.
     *
     * Nota académica: el uso de un bloque estático simplifica el ejemplo; en
     * aplicaciones
     * complejas la configuración del pool suele centralizarse en un módulo
     * de inicialización
     * o en la DI del contenedor.
     */
    static {
        try {
            Properties props = new Properties();

            // try-with-resources para cerrar automáticamente el InputStream
            // Se utiliza ConexionBD.class.getClassLoader() para obtener el
            classloader.
            try (InputStream input =
                DatabaseConfig.class.getClassLoader().getResourceAsStream("db.properties")) {
                // Carga de las propiedades desde el fichero de
                configuración.
            }
        } catch (Exception e) {
            // Manejo de excepciones
        }
    }
}
```

```

        // Se espera que 'db.url', 'db.user' y 'db.password' estén
presentes.
        props.load(input);
    }

    // Configuración mínima de HikariCP. Se explican los parámetros
relevantes.
    HikariConfig config = new HikariConfig();
    config.setJdbcUrl(props.getProperty("db.url"));           // URL
JDBC, p.ej. jdbc:mysql://host:3306/schema
    config.setUsername(props.getProperty("db.user"));         //
Usuario de la BD
    config.setPassword(props.getProperty("db.password"));     //
Contraseña de la BD

    // Parámetros de tuning básicos (ejemplo didáctico)
    config.setMaximumPoolSize(5);           // Número máximo de conexiones
activas en el pool
    config.setMinimumIdle(2);               // Número mínimo de conexiones
inactivas a mantener
    config.setIdleTimeout(10000);           // Tiempo (ms) tras el cual una
conexión inactiva puede cerrarse
    config.setConnectionTimeout(10000);    // Tiempo (ms) máximo a
esperar por una conexión libre
    config.setPoolName("DAMPool");         // Nombre del pool para
facilitar diagnóstico

    // Creación del DataSource (pool) a partir de la configuración
anterior.
    dataSource = new HikariDataSource(config);
    System.out.println("Pool de conexiones HikariCP inicializado
correctamente.");
    } catch (Exception e) {
        /*
        * En este ejemplo didáctico se lanza una RuntimeException para
detener el
        * arranque si el pool no puede inicializarse. En entornos
productivos se
        * recomienda un manejo más fino (log, métricas, reintento,
fallback).
        */
        throw new RuntimeException("Error al inicializar el pool de
conexiones", e);
    }
}

/**
 * Devuelve una Connection obtenida del pool.
 * <p>
 * Comportamiento pedagógico:
 * - La Connection debe cerrarse por el consumidor cuando deje de usarse.
 * - En un pool, Connection.close() no cierra la conexión física, sino
que la devuelve al pool.
 *
 * @return Connection del pool HikariCP
 * @throws SQLException si no es posible obtener la conexión
 */
public static Connection getConexion() throws SQLException {
    return dataSource.getConnection();
}

```

```

    }

    /**
     * Cierra el HikariDataSource liberando recursos asociados (conexiones
     físicas, hilos, ...).
     * <p>
     * Uso recomendado:
     * - Invocar al finalizar la aplicación, por ejemplo en un shutdown hook
     o en el lifecycle
     * del contenedor que despliegue la aplicación.
     */
    public static void cerrarPool() {
        if (dataSource != null && !dataSource.isClosed()) {
            dataSource.close();
            System.out.println("Pool de conexiones cerrado.");
        }
    }
}

```

Parte 3: Explicación de Decisiones Técnicas

1. ¿Por qué usamos PreparedStatement?

Con PreparedStatement evitas que el usuario pueda meter código malicioso, porque los datos van aparte y no se pueden "colar" comandos raros. Además, la base de datos puede guardar la consulta preparada y así va más rápido si la usas muchas veces. Es la forma recomendada cuando trabajas con datos de usuario.

2. ¿Ventajas del pool de conexiones?

En vez de estar abriendo y cerrando una conexión para cada usuario, se abren unas pocas y se van reutilizando. Eso hace que la aplicación sea mucho más rápida, gaste menos recursos y no se sobrecargue la base de datos cuando hay muchos accesos a la vez.

3. ¿Cuándo usar procedimientos almacenados?

Son útiles cuando necesitas repetir una operación compleja varias veces, como asignar empleados a proyectos o hacer cálculos grandes. Así la lógica se queda en la base de datos y no tienes que mandar tantos datos de un lado a otro. También te ahorra trabajo si la misma operación la usan varios programas, porque solo tienes que cambiarla en un sitio.

4. ¿Por qué controlar transacciones?

Porque con las transacciones te aseguras de que todas las partes de una operación se hacen correctamente, y si falla algo, se deshace todo. Así nunca tienes datos a medias, por ejemplo, un empleado con salario modificado pero el proyecto sin descontar el dinero. Te ayuda a mantener la base de datos siempre correcta.

Parte 4: Problemas Encontrados y Soluciones

Uno de los problemas que tuve es a la hora de hacer el menú controlar de que el usuario al meter una letra en vez de un número no me hiciera explotar el programa. Lo solucioné escribiendo esta línea de código.

```
opcion = entrada.hasNextInt() ? entrada.nextInt()
```

De normal recogeríamos el dato numérico con el nextInt y luego limpiaríamos el buffer, pero reemplazándolo con esas líneas, podemos controlar la entrada de letras en el menú.

Otro problema que he tenido es que a la hora de hacer el CRUD en el DAO, no se me aplicaban los cambios y era porque no estaba llamando por el mismo nombre a los datos de la base de datos.

```
Empleado emp = new Empleado(  
    rs.getInt( columnLabel: "id_empleado"),  
    rs.getString( columnLabel: "nombre"),  
    rs.getString( columnLabel: "departamento"),  
    rs.getBigDecimal( columnLabel: "salario"),  
    rs.getDate( columnLabel: "fecha_contratacion").toLocalDate(),  
    rs.getBoolean( columnLabel: "activo")
```

Otro problema que me dio, es que al hacer que se incrementase el salario y descontase el presupuesto con una sola operación, si había un error en una de las actualizaciones, los datos se quedaban mal porque no estaba usando las transacciones, para ello, lo que hice fue poner el setAutoCommit(false) y use el commit únicamente cuando ambas operaciones daban bien, en caso de que hubiese algún error utilizaba un rollback para que la base de datos se quedara limpia.